

1.1 | OPERATING SYSTEM

An operating system runs on computer hardware and serves as a **platform** for other softwares to run on the computer system.

An operating system is a program (system software) which acts as an interface between user and computer hardware.

A Computer system can be logically divided into **four** components:

Computer Hardware, Operating system, Application programs and Users.

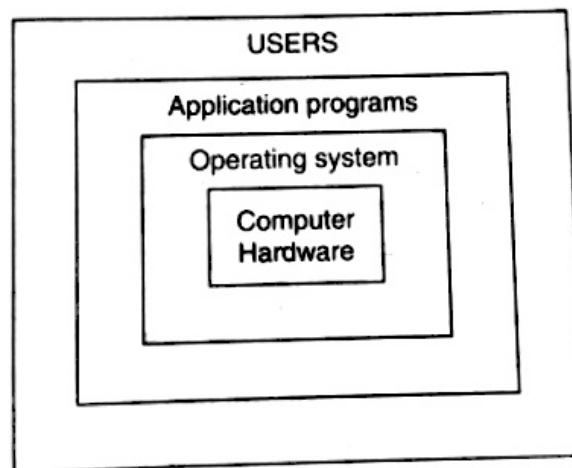


Fig. 1.1 Logical computer hierarchy

An operating system **controls** and **coordinates** the hardware components (**CPU, Memory, I/O devices**) among various application programs for different users.

An operating system has two main **objectives**:

- **Convenience**: An operating system makes a computer more convenient to use.
- **Efficiency**: An operating system allows the computer resources to be used in an efficient manner.

An Operating system is *similar to a government*. Like a Government it performs no useful work by itself. It simply provides an **environment** within which other programs can do useful work.

Operating system is also called a **Resource Manager**. Operating system **manages** all computer resources and **allocates** them to a specific program and **use** as it require to complete its task. So they are also called **Resource Allocator**.

Operating system is also called a **control program** as it controls the execution of user program to prevent errors and improper use of computers.

A user reads a file in English but it is stored in the form of **0** and **1** in the memory.

The internal details listed below are completely **unknown** to user and he simply **opens** and **reads** a file by typing a command through **editor**. These details are:

- (i) Structure of a file
- (ii) The way in which it is stored in a secondary memory.

The operating system **hides** all the **innermost** details of the working of computer hardware components and provides a **simple, usable and effective model** of a computer. This type of image of a computer is called **virtual machine**.

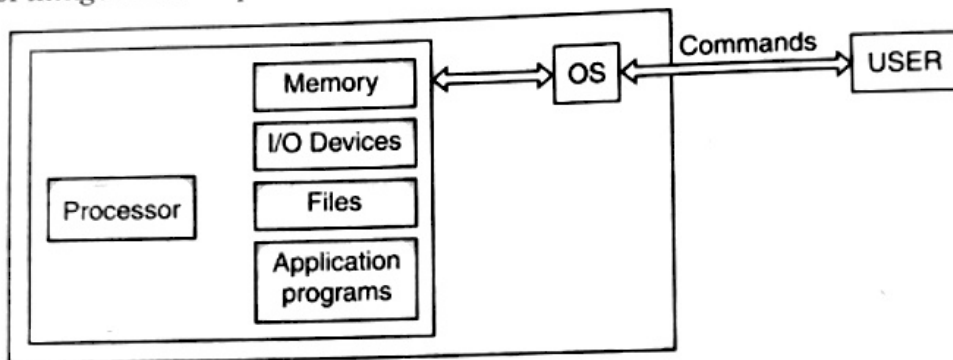


Fig. 1.2 Virtual computer = Computer hardware + Operating system

The common operating systems are the **windows family** of operating systems (viz., Windows 95, 98, 2000, NT, Vista), the **UNIX family** of operating systems (which includes UNIX, LINUX and many other derivatives) and the **Macintosh operating system**. Many other OS are also available for special purpose applications including specializations for **mainframes, robotics, real-time systems** and so on.

1.2 | HOW AN OPERATING SYSTEM TAKES CONTROL?

When we switch on our computer system certain logical steps take place after this computer loads the operating system and passes the control to it. This process is known as **booting**. These steps are shown in following figure 1.3.

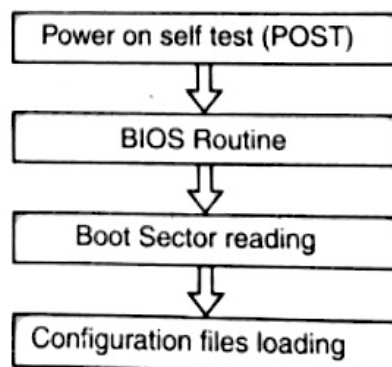


Fig. 1.3 Booting process

As shown in figure 1.3:

- (i) Booting process begins with Power on Self Test (**POST**). In this step, computer checks itself to see if the basic devices (viz. keyboard, mouse, printer, video etc.) are connected with the processor. For this it sends predefined signals to these devices and expects pre-defined responses back from these devices.
- (ii) Then Basic Input Output System (**BIOS**), which is a part of computer's Read Only Memory (**ROM**) performs checking the memory of the computer.
- (iii) Then BIOS reads the disk sector where files are needed to start the operating system. The area of the disk where these files are located is called **boot sector**. These files are very important for loading the operating system. If they are not present, the computer cannot start its working.
- (iv) Once these files are loaded, few more configuration files are also required which contain what to do upon the start up of the computer.

After fourth stage, booting process is completed and now operating system takes control of the computer.

1.3 | IS AN OPERATING SYSTEM ESSENTIAL TO USE A COMPUTER?

It is **not essential** to have an operating system to use a computer. Some special purpose computers do not have an operating system. But in each computer the duties of the operating system are taken by the application code.

A computer system has the ability to run various programs. If we are not using operating system then each application performed on computer would have to program its own operating system services. An operating system is useful because it contains a library of reusable services.

The operating system implements many necessary features that are used by nearly every program. So, **it is a good approach to implement them once**, in the operating system, rather than in each application program.

1.4 | FUNCTIONS OF AN OPERATING SYSTEM

An operating system manages four major resources of the computer system: **memory, processor, Input/output devices and files**. So, an operating system provides four operations:

- (i) **Memory Management function** finds free space in memory and allocate it to different processes.
- (ii) **Processor Management function** allocates the processor to execute a chosen process.
- (iii) **I/O Device Management function** allocates a device to a process.
- (iv) **File Management function** keeps track of all informations about files that how they are opened or closed.

NOTES:

1. **Process:** A process is a program at the time of its execution. Sometimes it is called a **task**.
2. **Jobs:** A job is a collection of activities needed to do the required work. A job can be divided into smaller steps named **job steps** and they have to be executed sequentially.

- **Job scheduler** selects a jobs from the job queue submitted to the system for execution.
 - **Process scheduler** decides when a process is to be executed in the case of multiprogramming.
 - **Dispatcher** allocates the processor for a particular process which is chosen by process scheduler.
 - **I/O traffic controller** is a program which constantly keeps the track of I/O devices, channels, control processors.
 - **I/O Scheduler** decides which device is to be allocated to which process.
 - **File system** is a collection of functions which are used to find status, usage and
-

location of a file. From this all informations can be easily find which are stored on a disk in the form of files.

1.4.1 Memory Management

■ **Primary (Main) Memory**

- Provide direct access storage for CPU.
- Processes must be in main memory to execute.

■ **Operating System must:**

- Policies (What should be done?)
 - Decide when to load each process into memory
 - Decide how much memory space to allocate each process.
 - Decide when a process should be removed from memory.
- Mechanism (How it should be done?)
 - Keep track of memory in use
 - Keep track of unused (free) memory
 - Protect memory space
 - Allocate, deallocate spaces for processes.
 - Swap processes: memory $\longleftrightarrow$ disk

1.4.2 Process Management

■ Operating system **manages** many kind of activities:

- User programs
- System programs: printer spoolers, name servers, file servers, etc.

■ Each is **encapsulated** in a process

- A process includes the complete execution context (code, data, PC registers, Operating system resources inuse, etc.).
- A process is **not** a program.
 - A process is one instance of a program at the time of its execution. Many processes can be running the same program. (example 4.1)

■ **Operating system must:**

- Create, delete, suspend, resume, and schedule processes.
- Supports **inter-process communication** and **synchronization**, **handle deadlock**.

1.4.3 Disk Management

■ **Disk**

- The actual hardware that sits underneath the file system.
- Large enough to store all user programs and data, application programs, entire OS.
- Persistent: recover from system failures.

■ **Operating system must:**

- Manage disk space at low level:
 - Keep track of used spaces

- Keep track of unused (free) space
- Keep track of "bad sectors"
- Handle low level disk functions such as:
 - Scheduling of disk operations
 - Head Movement.

1.4.4 File System Management

■ File system

- Disks (secondary storage) provide long term storage, but are unable to use directly.
- File system provides files and various operations on files.
 - A file is a long term storage entity, a named collection of persistent information that can be read or written.
 - A file system supports **directories**, which contain files and other directories—Name, size, data created, data last modified, owner etc.

■ Operating system must:

- Create and delete files and directories
- Manipulate files and directories
 - Read, write, extend, rename, copy, protect
- Provide general higher-level services
 - Backup, accounting, quotas.

1.5 | EVOLUTION OF OPERATING SYSTEM

To understand the key requirements for an OS and the significance of the major features of an OS, it is useful to consider how OS have evolved over the years. Following subsections illustrates the evolution of operating system :

1.5.1 Serial Processing

Earlier computer from late 1940s to the mid-1950s, the programmer intricated directly with the computer hardware. There was no operating system. These machines are called as **Bare Machine**. Resources of these systems are dedicated to a **single program** until its completion. Every computer system is programed in its **machine language** which was a cumbersome job. Programs were developed by manually translating sequences of instructions into binary or some other code whose base is **integral power of 2**. Programs are started by loading the program counter (**PC**) with the base address (address of first instruction). Data was entered through **hexadecimal keyboard**. Results can be obtained by examining the contents of registers and memory locations. As a result, programming on Bare machine results **low productivity**.

Next evolutionary step of computer system come by the usage of Input/output devices such as **punch cards, paper tape and language translators**. Programs are now translated into executable coded form called as **Loader** that automates the process of loading executable programs into memory. These early systems presented following two **major problems**:

(i) **Scheduling**: Most installations used a sign up sheet to reserve machine time. A user may sign up for an hour but finishes his jobs in 45 minutes. This would result in

wasted computer idle time. Also, the user might run into the problem not finish his job in the allotted time of an hour, and be forced to stop before the completion of his job.

(ii) **Set up Time:** A single program involves loading the compiler and the source program into memory, saving the compiled program (**object program**) and then loading and linking together the object program and common functions. Each of these steps could involve of mounting or dismounting tapes on setting up punch cards. If an error occurred, user had to go back to the beginning of the set up sequence. Thus, a considerable amount of time was spent just in setting up the program to run.

This mode of operation is turned as serial processing, reflecting the fact that users access the computer in series. After sometimes various software tools were developed to make serial processing more efficient. These included libraries of common functions, linkers, loaders, debuggers, I/O derives routines that were available as a **software** for users.

1.5.2 Batch Processing

Early machines were very **expensive** and so it was important to maximize the machine use. The wasted time caused by scheduling and set up time was unacceptable.

To improve the usage, the concept of batch processing was evolved. **Batch** is defined as a group of jobs with similar needs. The operating system allows user to form batches. Computer executes each batch **sequentially**, processing all jobs of a batch considering them as a single process is called **batch processing**.

The central idea behind the batch processing scheme was the use of a software called **Monitor**. With the use of this type of operating system, user does not have the direct access to the machine. Rather, user submits the job on cards or tapes to a computer operator who batches the jobs together sequentially and places the entire batch on an input device for the use of monitor. Each program is constructed to branch back to the monitor when it completes processing, at which point the monitor automatically begins loading the next program.

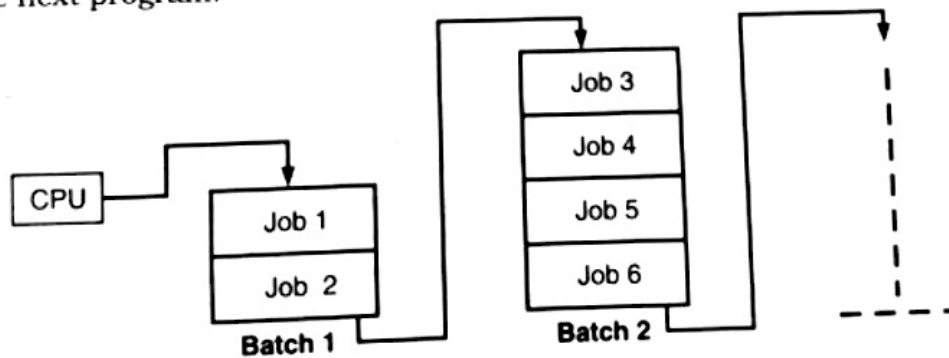


Fig. 1.5 Batches used in batch processing

In this execution environment, CPU is often idle, because the speed of mechanical I/O devices is **slower** than electronic device CPU.

In order to overcome the problem of speed mismatch the concept of **spooling*** was introduced. After sometimes, improvement in technology and introduction of disks resulted in faster I/O devices. However, CPU speed is increased even a greater extent. So the problem was not resolved.

The introduction of disk technology allows the operating system to keep all jobs on a disk, rather than on a serial card reader. With direct access on jobs operating system performs **jobs scheduling** to accomplish its task efficiently.

➤ **Spooling:** This acronym stands for **simultaneous peripheral operations online**. This 'absorbs' surplus processor time by performing I/O transfer for other jobs. I/O data is routed via disk files so that these jobs were only required to communicate with disk systems which are more faster. A set up of spooling system is illustrated in figure 1.6.

Spooling was one of the earliest **multiprogramming** application. Jobs are read via a card reader and copied into a disk file. The operating system reads this file. Printable output generated by the running programs is written to another disk file, while a printing program constantly prints the data in this file. Another benefit of spooling is that access to the slow peripherals is restricted to a program named spool print program. This facilitates the sharing of devices such as printers between several running programs, avoiding the need for these programs to compete for possession of the printer.

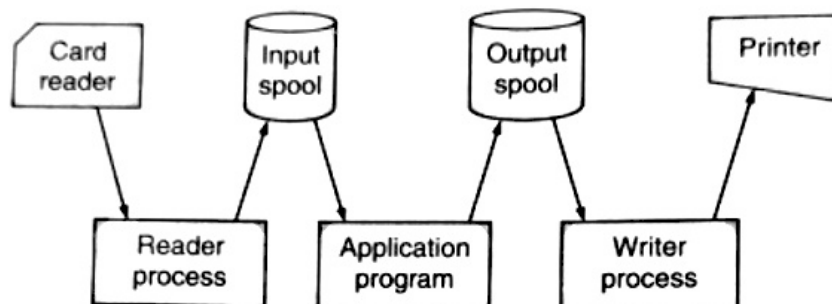


Fig. 1.6 Spooling system

Spooling can improve the performance of a computer system since it appears to create additional work in transferring data. Spooling operations run when no other jobs are available and hence use processor time which would otherwise be lost, while the real I/O incurred by the jobs use the much faster disk system. Overall the **throughput** of the system is improved so this technique is still important in modern day computers.

1.5.3 Multiprogramming

A single program cannot keep either CPU or I/O devices busy at all times. Multiprogramming increases CPU utilization by organizing jobs in such a manner that CPU has always one job to execute.

If computer is required to run several programs at the 'same time', the processor could be kept busy for most of the time by switching its attention from one program to the next. Additionally I/O transfers could overlap the processor activity i.e., while one program is awaiting for an I/O transfer, another program can use the processor. So CPU never sits idle or if comes in idle state then after a very small time it is again busy. This is illustrated in figure 1.7(a).

Multiprogramming is the **concurrent execution** of two or more processes. For example, considering two jobs J_1 and J_2 such that:

Job	Computation time	I/O time
J_1	$t(C_1)$	$t(I_1)$
J_2	$t(C_2)$	$t(I_2)$